5 MATRICES

Learning outcomes

By the end of this topic you should be able to:

5.1 Understand basic matrix **terminology**, and perform **matrix addition** and **subtraction** 5-1
5.2 **Multiply** one matrix by another (where possible) 5-7
5.3 Determine the **inverse** of a 2×2 matrix (where possible) 5-10
5.4 Use matrices to **solve a set of simultaneous equations** 5-12

5.1 Understand basic matrix **terminology**, and perform **matrix addition** and **subtraction**

MATRIX

A **matrix A** is an array of numbers, usually presented in the form:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

ORDER

The m horizontal lists of numbers are called the **rows** of **A**.
The n vertical lists of numbers are called the **columns** of **A**.
The **order** of a matrix with m rows and n columns is $m \times n$.

A matrix is usually represented by a capital letter in bold, e.g. **A**.
a_{ij} is used to represent the ij-th entry in a matrix **A**, i.e. the number that appears in row i and column j.

VECTOR

A **vector** is a matrix with only one column, or only one column.
A vector with only one row is called a **row vector**.
A vector with only one column is called a **column vector**.

TRANSPOSE

The **transpose** of a matrix **A**, written $\mathbf{A}^T$, is the matrix obtained by writing the rows of **A**, in order, as columns.
So if **A** has order $m \times n$, then $\mathbf{A}^T$ has order $n \times m$.

SYMMETRIC MATRIX

A matrix is **symmetric** if $\mathbf{A}^T = \mathbf{A}$

SQUARE MATRIX

A **square matrix** has the same number of rows as columns, e.g. a 3×3 matrix is a square matrix.

LEADING DIAGONAL

For a square matrix, the diagonal of numbers from top left to bottom right is called the **leading diagonal**. It is also referred to as the principal diagonal or main diagonal.

Author: Glenn Hawe © Ulster University 2024

IDENTITY MATRIX

The **identity matrix** is a square matrix which has ones along the leading diagonal, and zeroes everywhere else. The $n \times n$ identity matrix is denoted by $\mathbf{I}_n$. For example:

$$\mathbf{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

MATRIX OF ONES

A **matrix of ones** is a matrix where every element is equal to one. Examples of such matrices are:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \qquad \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \qquad \begin{pmatrix} 1 & 1 \end{pmatrix}$$

ADDITION AND SUBTRACTION OF MATRICES

Two matrices $\mathbf{A}$ and $\mathbf{B}$ can only be added or subtracted if they have the same order. $\mathbf{A}+\mathbf{B}$ is the matrix obtained by adding corresponding elements from $\mathbf{A}$ and $\mathbf{B}$. i.e. if $\mathbf{C} = \mathbf{A}+\mathbf{B}$ then $c_{ij} = a_{ij} + b_{ij}$. $\mathbf{A}-\mathbf{B}$ is the matrix obtained by subtracting the elements of $\mathbf{B}$ from the corresponding elements of $\mathbf{A}$. If $\mathbf{C} = \mathbf{A}-\mathbf{B}$ then $c_{ij} = a_{ij} - b_{ij}$

MULTIPLICATION OF A MATRIX BY A SCALAR

To multiply a matrix $\mathbf{A}$ by a scalar $k \in \mathbb{R}$, we multiply each element of $\mathbf{A}$ by k. Each element of $k\mathbf{A}$ is ka_{ij}.

Worked Examples

EXAMPLE 5.1

Let:

$$\mathbf{A} = \begin{pmatrix} 1 & 4 \\ 3 & -1 \\ 0 & 7 \end{pmatrix} \qquad \mathbf{B} = \begin{pmatrix} -2 & 1 & 3 \\ -5 & 6 & 1 \end{pmatrix} \qquad \mathbf{C} = \begin{pmatrix} -1 & -2 \\ 7 & 2 \end{pmatrix}$$

5.1.1 What is the order of $\mathbf{A}$?

Solution

The order of $\mathbf{A}$ is 3×2.

5.1.2 What is the value of b_{12}?

Solution

$b_{12} = 1$ (the number in row 1, column 2 of $\mathbf{B}$).

5.1.3 What is the value of $5\mathbf{C}$?

Solution

$$5\mathbf{C} = \begin{pmatrix} 5 \times -1 & 5 \times -2 \\ 5 \times 7 & 5 \times 2 \end{pmatrix} = \begin{pmatrix} -5 & -10 \\ 35 & 10 \end{pmatrix}.$$

5.1.4 What is $\mathbf{A} + \mathbf{B}^{\mathsf{T}}$?

Solution

$$\begin{pmatrix} 1 & 4 \\ 3 & -1 \\ 0 & 7 \end{pmatrix} + \begin{pmatrix} -2 & -5 \\ 1 & 6 \\ 3 & 1 \end{pmatrix} = \begin{pmatrix} -1 & -1 \\ 4 & 5 \\ 3 & 8 \end{pmatrix}$$

Python Code Examples

Python Example 5.1: Create a matrix

The code below shows how to create the matrix

$$\mathbf{A} = \begin{pmatrix} 2 & 1 & 3 \\ -4 & 5 & 7 \end{pmatrix}$$

Python	Output
`import numpy as np`	`A =`
`A = np.array([[2,1,3],[-4,5,7]])`	`[[ 2  1  3]`
`print("A = \n",A)`	`[-4  5  7]]`

Python Example 5.2: Access individual elements of a matrix

The code below shows how to access a particular element of a matrix, in this case a_{22} of **A**. Note that indexing begins at zero in Python. Therefore A[0,0] is the number in the first row and first column of **A**, i.e. a_{11}.

Python	Output
`print("a_22 =", A[1,1])`	`a_22 = 5`
`print("a_11 =", A[0,0])`	`a_11 = 2`

Python Example 5.3: Access a row of a matrix

Below is code to access the second row of **A**.

Python	Output
`print('Row two of A')`	`Row two of A`
`print(A[1,:])`	`[-4  5  7]`

Python Example 5.4: Access a row of a matrix

Below is code illustrating how to access specific elements of a row. Here we access the first two elements of the second row of **A**. Note that we use 0: 2 to refer to the first two columns, i.e. indices 0 and 1.

Python	Output
`print(A[1,0:2])`	`[-4  5]`

Python Example 5.5: The meaning of negative indices

The index '-1' is a shortcut for the last index.
The index '-2' is a shortcut for the second last index, and so on.

Python

```python
#row 0 (i.e. first row), last column

print("A[0,-1] =", A[0][-1])

#last row, last column

print("A[-1,-1] =", A[-1][-1])

#row 0 (i.e. first row), second last column

print("A[0,-2] =", A[0][-2])
```

Output

```
A[0,-1] = 3

A[-1,-1] = 7

A[0,-2] = 1
```

Python Example 5.6: Display the order of a matrix

The code below shows how to print the order of matrix **A**.

Python

```python
print('A has order',A.shape)
```

Output

```
A has order (2, 3)
```

Python Example 5.7: Take the transpose of a matrix

The code below shows how to take the transpose of a matrix **A**.

Python

```python
print('Transpose of A')

print(A.T)
```

Output

```
Transpose of A

[[ 2 -4]

 [ 1  5]

 [ 3  7]]
```

Python Example 5.8: Create an column vector using transpose

Writing a column vector can involve a lot of square brackets (one pair per row, each around a single number). Transposing a row vector is a more convenient way of creating a row vector. Below we show both ways for creating the column vector

$$\mathbf{y} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

Python

```python
#Creating a column vector normally

y = np.array([[1],[2],[3]])

print(y)


# Creating a column vector using transpose

y = np.array([[1,2,3]]).T

print('\ny =\n',y)

print(y.shape)
```

Output

```
[[1]
 [2]
 [3]]


y =
 [[1]
 [2]
 [3]]
(3, 1)
```

Python Example 5.9: Create an identity matrix

We can create an identity matrix using `np.eye`. Below we create the 3×3 identity matrix

$$\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Python

```python
# Create a 3 x 3 identity matrix

I = np.eye(3)

print('\nI =\n',I)
```

Output

```
I =
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Python Example 5.10: Create an all-ones matrix

We can create a matrix of ones using `np.ones`. Below we create a 2×3 matrix of ones.

Python

```python
B = np.ones((2,3))

print('B =\n',B)
```

Output

```
B =

 [[1. 1. 1.]

 [1. 1. 1.]]
```

Python Example 5.11: Add two matrices

Matrices **A** and **B** have the same order, so they can be added together as follows:

Python

```python
# Add A to B

print('A + B =')

print(A + B)
```

Output

```
A + B =

[[ 3.  2.  4.]

 [-3.  6.  8.]]
```

Python Example 5.12: Restrict operations to specific elements

We can restrict operations such as addition to specific elements of a matrix. Below we add 1 to each element in the first two columns of **A**, and print the resulting matrix.

Python

```python
A[:,0:2] = A[:,0:2] + 1

print('Updated matrix')

print(A)
```

Output

```
Updated matrix

[[ 3  2  3]

 [-3  6  7]]
```

Python Example 5.13: Multiply a matrix by a scalar

We can multiply a matrix by a scalar as follows. Below we multiply **B** by 5.

Python

```python
print('5B')

print(5*B)
```

Output

```
5B

[[5. 5. 5.]

 [5. 5. 5.]]
```

5.2 **Multiply** one matrix by another (where possible)

MATRIX-MATRIX MULTIPLICATION

Consider the $n \times p$ matrix **A**, and the $r \times q$ matrix **B**:

- The multiplication **AB** can only be carried out if $p = r$. i.e. if the number of columns of **A** equals the number of rows of **B**.
- The order of the resulting matrix is $n \times q$.

If the order of **A** is $n \times p$ and the order of **B** is $p \times q$, then the product **AB** gives a $n \times q$ matrix **C** whose elements are given by

$$c_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}$$

In words, to get the element in row i and column j of matrix $\mathbf{C} = \mathbf{AB}$:

- work along the entries in row i of matrix **A** $(a_{i1}, a_{i2}, a_{i3}, \ldots)$, and
- work down the entries in column j of matrix **B** $(b_{1j}, b_{2j}, b_{3j}, \ldots)$,
- adding together the products of corresponding entries $(a_{i1}b_{1j} + a_{i2}, b_{2j} + \ldots)$.

This is illustrated below for the calculation of c_{22}, the entry in the second row and second column of the product $\mathbf{C} = \mathbf{AB}$.

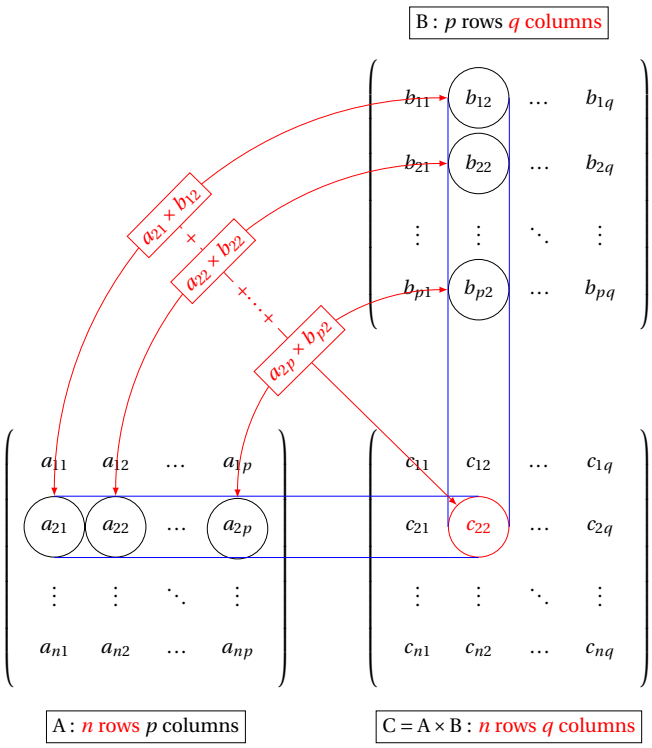

Figure from: http://www.texample.net/tikz/examples/matrix-multiplication/

INTEGER POWERS OF MATRICES

A square matrix **A** may be raised to integer powers, for example:

- $\mathbf{A}^2 = \mathbf{A} \times \mathbf{A}$
- $\mathbf{A}^3 = \mathbf{A} \times \mathbf{A} \times \mathbf{A}$

Worked Examples

EXAMPLE 5.2

Calculate $\mathbf{C} = \mathbf{AB}$, where $\mathbf{A}$ and $\mathbf{B}$ are as follows:

$$\mathbf{A} = \begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix}; \mathbf{B} = \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix}$$

Solution

The entry c_{11} in the first row and first column of $\mathbf{C} = \mathbf{AB}$ is given by summing the products of the entries in the first row of $\mathbf{A}$ with the entries of the first column of $\mathbf{B}$:

$$\begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{pmatrix}$$

$c_{11} = (2\times1)+(1\times-2)+(3\times0)+(4\times4) = 16$

The entry c_{12} in the first row and second column of $\mathbf{C} = \mathbf{AB}$ is given by summing the products of the entries in the first row of $\mathbf{A}$ with the entries of the second column of $\mathbf{B}$:

$$\begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{pmatrix}$$

$c_{12} = (2\times0)+(1\times2)+(3\times3)+(4\times6) = 35$

The entry c_{13} in the first row and third column of $\mathbf{C} = \mathbf{AB}$ is given by summing the products of the entries in the first row of $\mathbf{A}$ with the entries of the third column of $\mathbf{B}$:

$$\begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{pmatrix}$$

$c_{13} = (2\times3)+(1\times4)+(3\times5)+(4\times1) = 29$

The entry c_{21} in the second row and first column of $\mathbf{C} = \mathbf{AB}$ is given by summing the products of the entries in the second row of $\mathbf{A}$ with the entries of the first column of $\mathbf{B}$:

$$\begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{pmatrix}$$

$c_{21} = (-4\times1)+(5\times-2)+(7\times0)+(-1\times4) = -18$

The entry c_{22} in the second row and second column of $\mathbf{C} = \mathbf{AB}$ is given by summing the products of the entries in the second row of $\mathbf{A}$ with the entries of the second column of $\mathbf{B}$:

$$\begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{pmatrix}$$

$c_{22} = (-4\times0)+(5\times2)+(7\times3)+(-1\times6) = 25$

$c_{23} = (-4\times3)+(5\times4)+(7\times5)+(-1\times1) = 42$

So overall $\mathbf{C} = \begin{pmatrix} 16 & 35 & 29 \\ -18 & 25 & 42 \end{pmatrix}$

Classroom Exercises

EXERCISE 5.1

X and Y are matrices such that X has order 3×4 and Y has has order 4×8. What is the order of:

5.1.1

XY

5.1.2

matrix **Z**, given that **ZX** is a 2×4 matrix?

5.1.3

matrix **W**, given that **YW** and $\mathbf{XW}^T$ are both valid multiplications.

Python Code Examples

Python Example 5.14: Matrix multiplication

There are several methods for performing matrix-matrix multiplication in python. You should consult the documentation for the differences (however, there are none for the purposes of this module). There is also no significant difference in performance.
Below we calculate the product

$$\begin{pmatrix} 2 & 1 & 3 & 4 \\ -4 & 5 & 7 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 3 \\ -2 & 2 & 4 \\ 0 & 3 & 5 \\ 4 & 6 & 1 \end{pmatrix}$$

Python

```python
Q = np.array([[2,1,3,4],[-4,5,7,-1]])

R = np.array([[1,0,3],[-2,2,4],[0,3,5],[4,6,1]])


print('Q @ R')

print(Q @ R)


print('np.dot(Q,R)')

print(np.dot(Q,R))


print('np.matmul(Q,R)')

print(np.matmul(Q,R))
```

Output

```
Q @ R

[[ 16  35  29]

 [-18  25  42]]

np.dot(Q,R)

[[ 16  35  29]

 [-18  25  42]]

np.matmul(Q,R)

[[ 16  35  29]

 [-18  25  42]]
```

Python Example 5.15: Matrix powers

We can take powers of matrices using `np.linalg.matrix_power`.
Below we calculate $\mathbf{T}^4$ where $\mathbf{T} = \begin{pmatrix} 2 & -2 \\ 3 & -1 \end{pmatrix}$.

Python	Output
`T = np.array([[2,-2],[3,-1]])`	`[[ -2  14]`
`print(np.linalg.matrix_power(T,4))`	`[-21  19]]`

5.3 Determine the **inverse** of a 2×2 matrix (where possible)

INVERSE OF A MATRIX

The **inverse** of a square matrix $\mathbf{A}$ is the matrix $\mathbf{A}^{-1}$ such that $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix.

INVERTIBLE; NON-SINGULAR

A square matrix may or may not have an inverse.
If a matrix has an inverse, it is called **invertible** or **non-singular**.
If a matrix does not have an inverse, it is called **non-invertible** or **singular**.

DETERMINANT

Consider the 2×2 matrix $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$.

The **determinant** of $\mathbf{A}$, denoted $\det \mathbf{A}$ or $|\mathbf{A}|$, is the number $ad - bc$.
The inverse of $\mathbf{A}$ exists iff $|\mathbf{A}| \neq 0$.

INVERSE OF A 2×2 MATRIX

If $|\mathbf{A}| \neq 0$, then $\mathbf{A}^{-1} = \dfrac{1}{|\mathbf{A}|}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$.

Worked Examples

EXAMPLE 5.3

Determine the inverse of:

5.3.1

$\mathbf{A} = \begin{pmatrix} 5 & 3 \\ 4 & 8 \end{pmatrix}$

Solution

$|\mathbf{A}| = (5 \times 8) - (3 \times 4) = 40 - 12 = 28$. $\mathbf{A}^{-1} = \dfrac{1}{28}\begin{pmatrix} 8 & -3 \\ -4 & 5 \end{pmatrix}$

5.3.2

$\mathbf{B} = \begin{pmatrix} 3 & 6 \\ 2 & 4 \end{pmatrix}$

Solution

$|\mathbf{B}| = (3 \times 4) - (2 \times 6) = 12 - 12 = 0$. So $\mathbf{B}^{-1}$ does not exist.

Python Code Examples

Python Example 5.16: Matrix inverse and determinant

Below we calculate the determinant and inverse of $\mathbf{A} = \begin{pmatrix} 5 & 3 \\ 4 & 8 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 3 & 6 \\ 2 & 4 \end{pmatrix}$. These are the matrices from the previous worked examples - compare the answers.
Note that 'inv' may well generate an answer even for a singular matrix so we must be careful. We should check the determinant using 'det' and if it is very close to zero the matrix may be singular. (Because computer representations of numbers have a finite precision, there can be numerical errors so the determinant of a singular matrix might not come out as exactly zero.) When we compute an inverse we can always check the solution. For example, we can see below that $\mathbf{AA}^{-1} = \mathbf{I}$ but $\mathbf{BB}^{-1} \neq \mathbf{I}$.

Python

```python
A = np.array([[5,3],[4,8]])

B = np.array([[3,6],[2,4]])

print('Determinant of A')

print(np.linalg.det(A))

print('Determinant of B')

print(np.linalg.det(B))


# Inverse of A

A_inv = np.linalg.inv(A)

print('Inverse of A')

print(A_inv)

print('\nCheck A x A_inv = I')

print(A @ A_inv)


# Inverse of B

B_inv = np.linalg.inv(B)

print('\nInverse of B')

print(B_inv)

print('\nCheck B x B_inv = I')

print(B @ B_inv)
```

Output

```
Determinant of A

27.999999999999996

Determinant of B

6.66133814775094e-16

Inverse of A

[[ 0.28571429 -0.10714286]

 [-0.14285714  0.17857143]]


Check A x A_inv = I

[[ 1.00000000e+00  0.00000000e+00]

 [-2.22044605e-16  1.00000000e+00]]


Inverse of B

[[ 6.00479950e+15 -9.00719925e+15]

 [-3.00239975e+15  4.50359963e+15]]


Check B x B_inv = I

[[0. 0.]

 [0. 0.]]
```

5.4 Use matrices to **solve a set of simultaneous equations**

MATRIX FORMULATION OF SIMULTANEOUS EQUATIONS

Consider the simultaneous equations:

$$\begin{aligned} ax_1 + bx_2 &= y_1 \quad (1) \\ cx_1 + dx_2 &= y_2 \quad (2) \end{aligned}$$

where a, b, c, d, y_1, y_2 are constants (the values of which we know) and x_1, x_2 are variables whose value we need to determine.
We can rewrite Equations (1) and (2) using matrices as

$$\mathbf{Ax} = \mathbf{y} \quad (3)$$

where

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \qquad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$$

To solve for $\mathbf{x}$ we rearrange Equation (3) to be in the form $\mathbf{x} = \ldots$.
We do this by (left) multiplying both sides of Equation (3) by $\mathbf{A}^{-1}$:

$$\begin{aligned} \mathbf{A}^{-1}\mathbf{Ax} &= \mathbf{A}^{-1}\mathbf{y} \\ \mathbf{Ix} &= \mathbf{A}^{-1}\mathbf{y} && \text{because } \mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \\ \mathbf{x} &= \mathbf{A}^{-1}\mathbf{y} && \text{because } \mathbf{Ix} = \mathbf{x} \end{aligned}$$

Worked Examples

EXAMPLE 5.4

Using matrices, solve the simultaneous equations

$$\begin{aligned} 2x_1 + x_2 &= 3 \\ x_1 + 3x_2 &= -1 \end{aligned}$$

Solution Let

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \qquad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad \mathbf{y} = \begin{pmatrix} 3 \\ -1 \end{pmatrix}$$

Then $\mathbf{Ax} = \mathbf{y}$, so $\mathbf{x} = \mathbf{A}^{-1}\mathbf{y}$. Now check that $\mathbf{A}^{-1}$ exists:

$$|\mathbf{A}| = (2 \times 3) - (1 \times 1) = 6 - 1 = 5 \neq 0$$

So $\mathbf{A}^{-1}$ exists and is given by:

$$\mathbf{A}^{-1} = \frac{1}{|A|}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}$$

Using $\mathbf{x} = \mathbf{A}^{-1}\mathbf{y}$ we have:

$$\mathbf{x} = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 3 \\ -1 \end{pmatrix} = \frac{1}{5}\begin{pmatrix} (3 \times 3) + (-1 \times -1) \\ (-1 \times 3) + (2 \times -1) \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 10 \\ -5 \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

i.e. $x_1 = 2, x_2 = -1$

Python Code Examples

Python Example 5.18: Solving simultaneous equations

Below we determine the solution to the following equations, i.e. determine the values of x_1, x_2, x_3.

$$\begin{aligned} 2x_1 + 3x_2 + x_3 &= 1 \\ x_1 + 5x_2 + 3x_3 &= 4 \\ 7x_1 + x_2 + 2x_3 &= 2 \end{aligned}$$

We can represent this using matrices as $\mathbf{Ax} = \mathbf{y}$, where $\mathbf{A} = \begin{pmatrix} 2 & 3 & 1 \\ 1 & 5 & 3 \\ 7 & 1 & 2 \end{pmatrix}$, $\mathbf{y} = \begin{pmatrix} 1 \\ 4 \\ 2 \end{pmatrix}$, and $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$.

To calculate $\mathbf{x} = \mathbf{A}^{-1}\mathbf{y}$, we can use the function `solve` from `linalg`, as shown in the cell below. Note that as with the computation of inverses:
- we should check that $|\mathbf{A}| \neq 0$: if $|\mathbf{A}| = 0$ (or is extremely close to zero) then $\mathbf{A}$ is singular and there is no solution to the simultaneous equations
- once we have a solution $\mathbf{x}$ we can always use matrix multiplication to check that $\mathbf{A}$ times $\mathbf{x}$ does indeed give $\mathbf{y}$.

Python

```python
A = np.array([[2,3,1],[1,5,3],[7,1,2]])

y = np.array([[1,4,2]]).T

print('A\n',A)

print('y\n',y)


x = np.linalg.solve(A,y)

print('x')

print(x)


print('\nSo the solution is:')

print('x_1 = ',x[0,0])

print('x_2 = ',x[1,0])

print('x_3 = ',x[2,0])
```

Output

```
A

 [[2 3 1]

 [1 5 3]

 [7 1 2]]

y

 [[1]

 [4]

 [2]]

x

[[-0.13513514]

 [-0.08108108]

 [ 1.51351351]]


So the solution is:

x_1 =  -0.13513513513513514

x_2 =  -0.08108108108108114

x_3 =  1.5135135135135136
```

Further resources

Text books

- Chapter 8 of W. D. Wallis (2012), A Beginner's Guide to Discrete Mathematics, 2nd ed.

Youtube videos

Link	Learning outcome			
	5.1	5.2	5.3	5.4
Khan Academy	x			
Khan Academy	x			
patrickJMT	x			
patrickJMT		x		
patrickJMT		x		
Khan Academy		x		
Khan Academy		x		
slcmath@pc		powers		
Joe Mc Eneaney			x	
patrickJMT				x
patrickJMT				x